

INTERNAL AI SALES ASSISTANT

MNST · Engineering Handover

*A dispatcher-first RAG system for product Q&A, discovery, and sales-aid generation,
grounded entirely in an approved-document knowledge base.*

Prepared by

Nishita Shirwal

Repository

Nishita-MultiNanoSense/sales-ai-assistant (branch:
week-02-streamlit-skeleton)

Deployment

Streamlit Community Cloud + Neon Postgres

Table of Contents

- Table of Contents2
- Executive Summary3
- System Architecture4
- End-to-End Request Flow6
- Codebase Structure7
- Major Subsystems 12
- Application Walkthrough 14
- How to Run the Project 17
- How to Modify the System..... 18
- Known Limitations and Edge Cases 19
- Troubleshooting 20
- Evaluation and Testing 21
- Final Handover Checklist 23

Executive Summary

What It Does

The Internal AI Sales Assistant is an internal Streamlit tool for the MNST sales team. It answers product and application questions using only a curated set of approved company documents -- never general knowledge - and supports the surrounding sales workflow: capturing customer requirements, running pre-call discovery, and generating customer-facing comparison documents, all grounded in the same knowledge base and the same safety checks.

Who Uses It

- **Sales reps** -- four pages: Assistant (Q&A), Customer Requirements, Discovery Questions, Sales Aids.
- **Admins** -- one password-gated Admin page: document management, claims policy, feedback review, deal cleanup.

Current Status

Deployed and running on Streamlit Community Cloud with Neon Postgres as the durable backing store. All five pages have been implemented and have been exercised by hand through the real UI. 4 test files: 2 are regression tests against a specific confirmed bug each, 1 covers the Postgres connection-retry path, and 1 is a broader design-coverage suite for XLSX extraction rather than a single-bug test. A known document-extraction limitation is that complex multi-level or multi-section tables can misparse, and a DOCX table nested inside another table's cell can be silently dropped.

The One-Sentence Architecture

Every question first goes through the Product Index, which can answer structured questions directly (such as "Which products are portable?" or "What does this ordering code mean?"). Questions it cannot answer are passed to the RAG pipeline, where relevant documents are retrieved and the LLM generates a grounded response.

System Architecture

This is not a plain RAG app. It's a dispatcher-first hybrid: a deterministic layer answers what it can exactly and without an LLM call; everything else falls through to a retrieval + generation pipeline that applies product scoping, keyword boosting, and comparison-specific retrieval; and every output -- deterministic or generated -- passes through the same claim guardrail before reaching the rep.

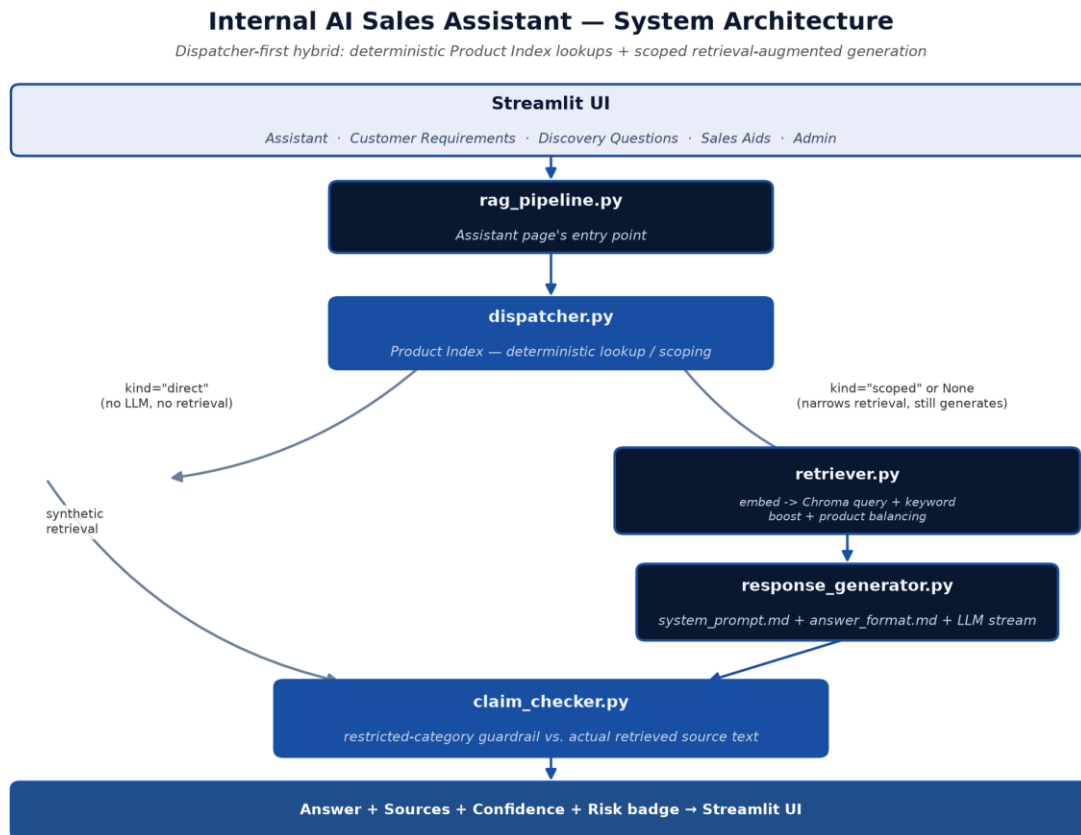


Figure 1. End-to-end pipeline for the Assistant page.

Layer by Layer

- **Application** -- one Streamlit multi-page app (st.navigation), five pages, no separate backend process.
- **RAG** -- two LLM-free preprocessing layers (dispatcher + retriever scoping) sit in front of a single system-prompt + user-message chat completion call. No agentic loop, no tool-calling.
- **Retrieval** -- ChromaDB local vector store, product-scoped filters, a literal keyword-boost pass, and a two-tier confidence threshold. The most heavily customized subsystem in the codebase.
- **Guardrail** -- a layered set of checks -- no-source refusal, ambiguous-product refusal, self-referential refusal, restricted-claim matching against real source text, and an ungrounded-email check.
- **UI** -- one committed visual design (the navy-sidebar/card look) shared by every page; app/theme.py applies its CSS unconditionally.

Discovery Questions and Sales Aids Are Separate Pipelines

They call the retriever and the LLM directly -- not through the Assistant's dispatcher/rag_pipeline path -- with their own prompts and their own document-type-based scoping. Customer Requirements has no LLM in its path at all; it's a pure form driven by the same Product Index.

End-to-End Request Flow

What happens, in order, when a rep asks a question on the Assistant page:

Step	What Happens
1. Intent classification	Keyword/regex only, informational -- shown to the LLM, doesn't route anything.
2. Dispatch	Checked against the Product Index for a matched product, an ordering code, or a catalog filter. A fully-resolved match skips retrieval and the LLM entirely.
3. Retrieval	If not fully resolved -- embeds the query, scopes to detected products, retrieves top-k (8 chunks) via cosine similarity, adds a keyword-boost pass, balances multi-product comparisons.
4. Pre-generation guardrails	No evidence -> canned refusal. Ambiguous product reference -> canned clarification. Self-referential question answered only from competitor content -> canned refusal. None of these call the LLM.
5. Generation	One streamed chat-completion call -- system prompt + numbered source excerpts + the question. A concentration-unit conversion, if asked, is computed in code and injected as a fact.
6. Finalization	Strips a fabricated, ungrounded email if present; runs the claim-checking guardrail against the finished answer; returns answer + sources + confidence + risk.
7. Display	Confidence/risk badges, a Sources expander, Correct/Wrong/Unsafe feedback buttons. Customer-facing wording is generated only on request, not eagerly.

Codebase Structure

Pages (app/*_page.py, app/deal_picker.py)

Module	Responsibility	Key functions / classes
streamlit_app.py	Entry point. Configures the page and theme, runs the one-time cold-start sync from Postgres and backend warm-up (both cached via st.cache_resource so they run once per server process, not per visitor), owns session-state init, renders the Assistant page directly, and builds the st.navigation page list -- wrapping every page in _safe_page() so an uncaught exception anywhere shows a plain refresh message instead of Streamlit's raw traceback.	render_assistant_page(), _ask_and_record(), _safe_page(), _read_doc_bytes()
admin_page.py	The password-gated Admin page. Six tabs: Documents (upload/type/remove/rename, each row now with a Download button), Approved Claims, Restricted Claims, Feedback (accuracy chart + report management), Deals, and Customer Requirements (form-field configuration). Slow actions (document removal, feedback clearing) run as background jobs so the admin isn't blocked.	render_admin_page(), _render_documents_tab(), _process_pending_doc_removal())
discovery_page.py	Pre-Call Discovery page. Two-stage workflow: generates documented "Right to Win" points with exploratory questions for a described use case, then -- once the rep has recorded enough of the customer's actual answers -- generates a grounded product recommendation. Also surfaces MEDDPICCC qualification questions for whichever of the 8 fields are still missing on the active deal.	render_discovery_page(), _render_right_to_win_card(), _render_qualification_card()
requirements_page.py	Customer Requirement Capture form. Product selection surfaces that specific product's real ordering options pulled live from the Product Index, not a generic field list, so a rep can never record a configuration the product doesn't actually support.	render_requirements_page(), _suggested_code(), _selectable_segments()
sales_aid_page.py	Sales Aid Generator page. Produces a short, customer-facing comparison document (MNST vs. a named or unnamed competitor) for a use case, with a .docx export that rebuilds the LLM's markdown comparison table as a real Word table.	render_sales_aid_page(), _sales_aid_as_docx()
deal_picker.py	The shared "which deal" selector used by Customer Requirements, Discovery, and Sales Aids. Owns session_state.active_deal_id so a deal picked on one page stays selected when a rep switches to another; starting a new deal is a single atomic form.	render_deal_picker(), set_active_deal()

Pipeline and Engine (everything else in app/)

Module	Responsibility	Key functions / classes
rag_pipeline.py	The Assistant page's only orchestrator. answer_question() / stream_answer_question() call dispatcher.py first, fall through to retriever.py + response_generator.py only if the dispatcher can't fully resolve the question, and merge in any nomenclature notes the dispatcher resolved. Also the reindex/add-doc/remove-doc CLI.	answer_question(), stream_answer_question(), finalize_streamed_answer()
dispatcher.py	Deterministic, LLM-free answer routing against the auto-extracted Product Index: exact-match product names, ordering-code decodes, and catalog-filter questions ("which products are portable") are answered directly here -- retrieval and the LLM are skipped entirely when fully resolved.	dispatch(), DispatchResult
retriever.py	Embeds queries and documents, builds/queries the Chroma vector index, and layers several custom behaviors on top of plain top-k search: product-name scoping, balanced multi-product retrieval for comparisons, a literal keyword-boost pass, and the none/low/high confidence classification.	retrieve(), build_index(), _keyword_boost_matches()
response_generator.py	All LLM call plumbing: loads/caches the system and format prompts, dispatches to OpenAI / Azure OpenAI / an offline mock depending on LLM_PROVIDER, streams the reply, and runs two deterministic checks on the finished text (a fabricated-email check and the claim guardrail) before returning a GeneratedAnswer.	stream_answer(), finalize_answer(), generate_customer_wording()
claim_checker.py	The restricted-claims guardrail: matches a question/answer against admin-configured keyword categories and verifies each match is actually backed by real retrieved source text -- not just an admin-typed Approved Claims bullet -- before it can be shown as customer-facing.	check_restricted_claims(), guardrail_source_text()
chunker.py	Turns a loaded document's blocks into retrieval-ready chunks, always respecting section/table boundaries so two unrelated topics never share a chunk, and assigns each document's product-name identity (with collision handling across a batch indexed together).	chunk_document(), assign_product_name(), chunk_approved_claims()

document_loader.py	Format-specific extraction for .docx/.pptx/.pdf/.csv/.xlsx/.md/.txt/images into one common block schema, including OCR for scanned pages and embedded images, three-style table classification, and two independent coverage checks that flag content the extractor may have dropped.	load_document(), extract_blocks(), check_table_structure()
registry_builder.py	Parses each approved .docx catalogue's raw structure to auto-extract the Product Index: decoded ordering-code segments, selectable specs, and the product's type/technology/install-type. Never guesses -- an unmatched field is recorded as "Unknown."	build_product_registry(), rebuild_product_registry()
product_index.py	Loads data/product_registry.json into typed dataclasses for the rest of the app to consume; the registry itself is fully regenerated by registry_builder.py on every reindex, never hand-edited.	load_product_index(), ProductIndexEntry, NomenclatureSegment
intent.py	Pure keyword/regex intent classifier. Informational only -- passed to the LLM as context, never used to route or gate anything downstream.	classify_intent(), INTENT_KEYWORDS
concentration.py	Hydrogen ppm / %v-v / %LEL unit conversion as exact, code-computed arithmetic, never left to the LLM -- a model can confidently compute a wrong conversion.	HydrogenConcentration, detect_conversion_request()
generation_helpers.py	Small shared utilities (context-block building, source deduplication, structured-reply section parsing) used by both *_generator.py modules so their similar "Header:\ncontent" parsing logic isn't duplicated.	build_context_block(), dedupe_sources(), split_sections()
discovery_generator.py	Generation logic behind the Discovery page: Right-to-Win point generation, MEDDPICCC qualification-question generation, and product recommendation generation. Calls retriever.py and the LLM directly, not through rag_pipeline.py.	stream_discovery_questions(), stream_recommendation()
sales_aid_generator.py	Generation logic behind the Sales Aid page: retrieves MNST and competitor content as two separate, guaranteed-represented passes so one side can't dominate the other, then generates the comparison document.	stream_sales_aid(), finalize_sales_aid()
db.py	The Postgres (Neon) connection layer every *_store.py module depends on: a pooled connection with retry-on-stale-connection logic (covered by a dedicated regression test) and the full table schema, created once per process.	get_connection(), ensure_schema(), DatabaseUnavailableError
theme.py	CSS injection (the navy-sidebar/card visual design, applied unconditionally) and centralized confidence/risk badge rendering, shared by every page.	inject_theme(), render_badges_html()

background_jobs.py	A minimal single-worker background-thread runner for slow admin actions, so the admin isn't stuck on a blocking spinner; status is polled by a sidebar fragment on a 3-second timer.	start_job(), get_active_jobs()
view_chroma.py	A tiny standalone debug CLI: prints the vector index's total chunk count and a preview of the first 5 chunks' metadata/text.	(script -- no importable functions)

Persistence Layer (app/*_store.py and related config modules)

All 11 of these follow the same pattern: each independently branches on `db.is_postgres_enabled()` to pick Postgres (Neon) vs. a local SQLite/JSON/YAML fallback, so local development never needs a live database connection.

Module	What it stores	Key functions
claims_store.py	One of the Admin page's six tabs. It allows an admin to add a short introductory text and individual approved facts directly, without editing an uploaded document. These claims are indexed and can supplement the catalogue knowledge base for ordinary questions. However, because they are admin-entered rather than sourced from an approved document, they are not treated as supporting evidence for restricted claims such as certifications, pricing, safety, delivery, or accuracy.	load_claims(), save_claims()
deals_store.py	Deal CRUD, the 8 MEDDPIC fields, and each deal's saved recommendation/discovery/qualification blob.	create_deal(), update_deal_fields(), missing_fields()
feedback_store.py	Correct/Wrong/Unsafe feedback CRUD and the daily-counts aggregation behind the Admin accuracy chart.	record_feedback(), daily_feedback_counts()
requirements_store.py	Customer Requirement submission CRUD, including the generic extra_fields JSON column for admin-added custom fields.	record_requirement(), list_requirements_for_deal() ()
sales_aid_store.py	Generated Sales Aid history -- one row per generation, never overwritten.	record_sales_aid(), list_sales_aids_for_deal()
document_storage.py	Uploaded document bytes in Postgres, restored to local disk on cold start.	save_document_bytes(), sync_local_docs_from_postgres()
document_types.py	Per-document admin-assigned type and the main-assistant-exclusion rule that keeps internal strategy documents out of rep-facing answers.	get_document_type(), is_excluded_from_main_assistant()
product_field_overrides.py	Admin edits to a specific product's selectable ordering fields (add/edit/hide) -- kept separate from product_registry.json since that file is fully regenerated on every reindex.	set_field(), effective_additional_params()

product_name_overrides.py	Admin correction of a document's auto-assigned product name, keyed by filename so it survives every future rebuild.	set_override(), load_overrides()
restricted_policy.py	The claim guardrail's own policy: categories, trigger keywords, and which terms are always-blocked regardless of source backing.	load_entries(), build_lookup()
requirements_fields.py	The Customer Requirements form's configurable field list, distinguishing core fields (tied to real typed database columns) from admin-added custom ones.	load_fields(), merge_core_fields()

data/ -- Knowledge Base and Config

Path	Contents
approved_docs/	13 real approved documents (.docx / .xlsx / .md) -- the actual knowledge base.
approved_claims.md	Admin-typed supplemental facts.
restricted_claims.yaml	The live guardrail policy.
product_registry.json	Auto-extracted Product Index -- regenerated on every reindex, never hand-edited.
*.db	SQLite local-dev fallback stores (used only when DATABASE_URL is unset).

Everything Else

- **scripts/seed_neon.py** -- one-time local-to-Postgres migration script, not run automatically.
- **tests/** -- 4 pytest files, all regression tests (see Evaluation & Testing).
- **.github/workflows/** -- two keep-alive cron pings (the app itself, and the Neon database).

Major Subsystems

1. Document Ingestion

Supports .docx, .pptx, .pdf, .csv, .xlsx, .md, .txt, and images (OCR-only). Table extraction is the most engineered part -- each table is classified into one of three styles and checked twice: a coverage check (did any cell silently disappear?) and a structure check (does the detected header look implausibly long -- the signature of a misdetected multi-row header?). Both warnings surface directly in the Admin UI on upload.

- **Chunking** -- boundary-respecting (never splits a section or a table mid-chunk), ~50 words per chunk. Approved Claims bullets are chunked one-bullet-per-chunk, a deliberate exception -- packing them together previously let one bullet's embedding get diluted by its neighbors (a real, fixed bug).
- **Product naming** -- derived from a document's own heading, with collision avoidance across a batch and an admin-override path that always wins and survives every future rebuild.
- **Reindexing** -- always a full rebuild from data/approved_docs/, so the index can never silently drift from what's actually approved.

2. Embeddings and Vector Store

- **Store** -- ChromaDB, local disk, single collection. Rebuilt from Postgres on cold start.
- **Embeddings** -- all-MiniLM-L6-v2 locally by default (OpenAI text-embedding-3-small optional).
- **Similarity** -- cosine, explicitly configured (Chroma defaults to L2 -- this is load-bearing, not a default).
- **Product Index** -- a separate, auto-extracted JSON (product_registry.json) with decoded ordering codes, selectable specs, and technology/type vocab. Extraction rule throughout: never guess -- an unmatched field is recorded as "Unknown."

3. Retrieval System

Never reduces to "retrieve the top-k chunks." Detected product mentions scope the vector search; 2+ products trigger a balanced, round-robin-interleaved retrieval instead of a single query (confirmed via real testing that one product otherwise dominates); a keyword-boost pass catches literal-term matches the embedding missed; and when a question names no product and no attribute, retrieval is deliberately balanced across every approved product rather than left to whichever ranks highest.

- **Confidence** -- two hardcoded similarity thresholds (0.32 / 0.56), both env-overridable.
- **No-match** -- returns an empty result, never raises -- the caller shows a canned refusal, no LLM call.

4. RAG / Response Generation

Seven separate prompt files, each purpose-built per output type (Assistant answers, customer-facing rewrites, discovery questions, qualification questions, product recommendations, sales aids).

REQUIRE_SOURCES (default on) makes "no relevant source" a hard pre-LLM gate. Customer-facing wording is gated by a global kill switch (ALLOW_CUSTOMER_FACING_OUTPUT, default off) plus the guardrail's own verdict, and is only ever generated on demand.

5. Claim Guardrail

The single most safety-critical subsystem -- covers every generated output, not just one page. Six restricted categories (Certification, Accuracy, Safety, Pricing, Delivery, Legal), admin-editable keyword policy with immediate effect. A key distinction: Approved Claims (admin-typed facts) can answer ordinary questions, but cannot by themselves satisfy a restricted-category claim -- that requires a real approved document as backing evidence.

6. Database and Persistence

Two independent stores: Postgres (Neon) for everything admin/rep-entered (documents, claims, deals, requirements, feedback, sales aids) when DATABASE_URL is set, with a full local-file/SQLite fallback for local dev; and a local Chroma vector index, always local, rebuilt from Postgres automatically on cold start. The connection layer retries a stale/suspended Neon connection before surfacing an error -- covered by a dedicated regression test.

7. Admin and UI

Password-gated Admin page with six tabs. Slow actions (document removal, feedback clearing) run as background jobs so the admin isn't stuck on a blocking spinner. Confirmation dialogs are written to do nothing but set a flag and rerun -- a documented upstream Streamlit bug means a dialog can otherwise get stuck open if any real work runs before the rerun.

Application Walkthrough

Captured from a live local run against the production knowledge base, for reference.

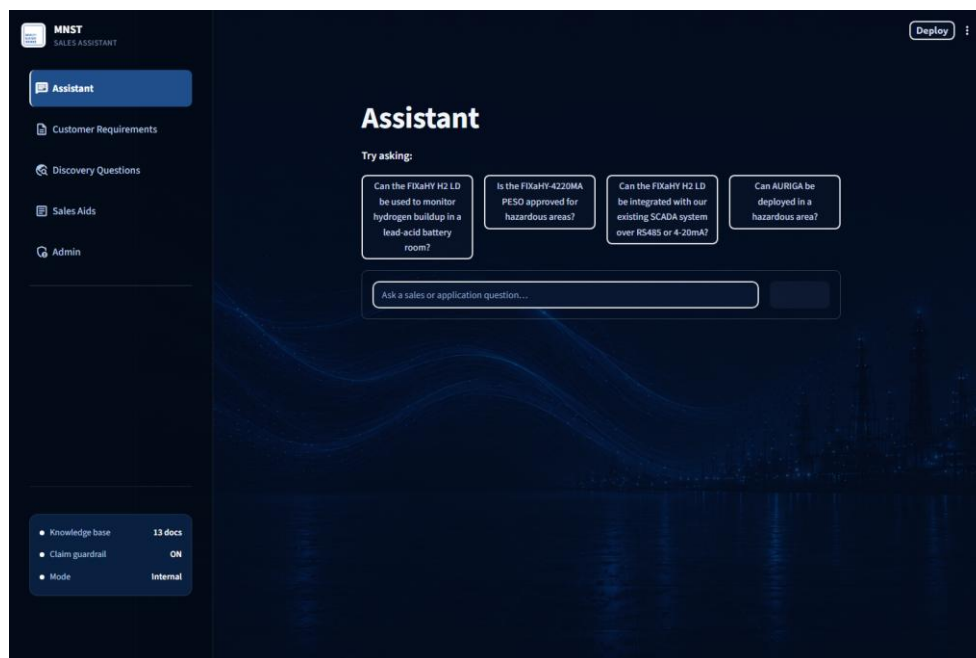


Figure 2. Assistant -- empty state, enterprise theme.

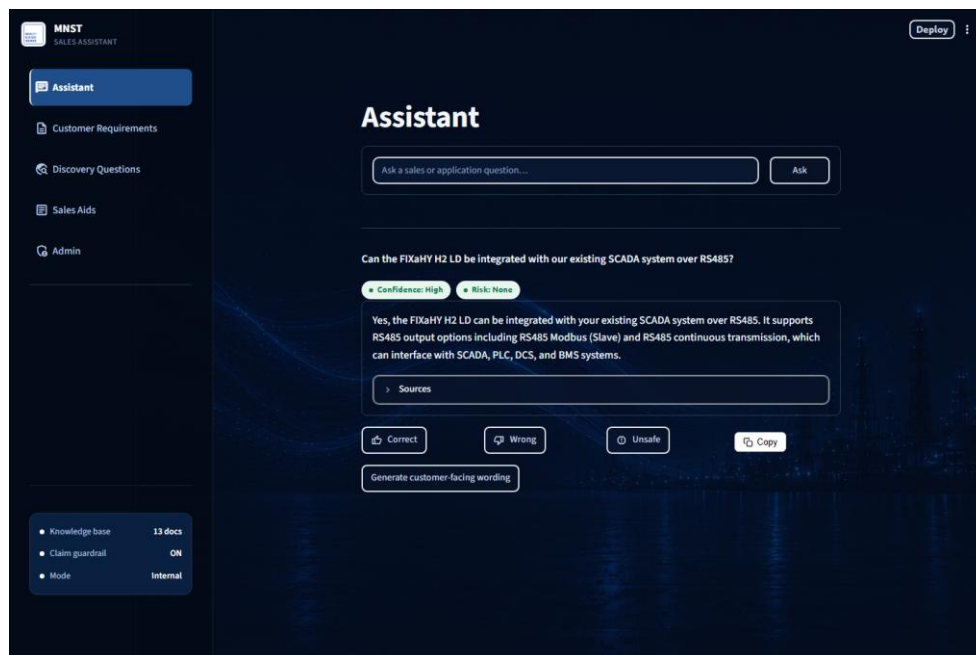


Figure 3. Assistant -- a real, grounded answer with confidence, risk, and sources.

MNST
SALES ASSISTANT

Assistant

Customer Requirements

Discovery Questions

Sales Aids

Admin

Deploy

Customer Requirement Capture

Capture a customer's technical requirements for a clean handoff to production.

Deal
+ New deal

Customer Name Company

Start New Deal

Pick an existing deal, or start a new one above (Customer Name + Company), to continue.

- Knowledge base 13 docs
- Claim guardrail ON
- Mode Internal

Figure 4. Customer Requirements -- Product-Index-driven intake form.

MNST
SALES ASSISTANT

Assistant

Customer Requirements

Discovery Questions

Sales Aids

Admin

Deploy

Pre-Call Discovery

Describe the customer's use case as you understand it so far. Get back documented Right-to-Win points -- each with its own evidence and exploratory questions -- not a product recommendation yet. Record the customer's answers during the call, then request a recommendation once you have enough of them.

Deal
+ New deal

Customer Name Company

Start New Deal

Pick an existing deal, or start a new one above (Customer Name + Company), to continue.

- Knowledge base 13 docs
- Claim guardrail ON
- Mode Internal

Figure 5. Discovery Questions -- deal picker and use-case capture.

MNST SALES ASSISTANT

Assistant
Customer Requirements
Discovery Questions
Sales Aids
Admin

Sales Aid Generator

Describe the customer's use case to get a short comparison document you can hand to the customer's technical champion -- grounded in the approved knowledge base only.

Deal
+ New deal

Customer Name
Company

Start New Deal

Pick an existing deal, or start a new one above (Customer Name + Company), to continue.

Knowledge base 13 docs
Claim guardrail ON
Mode Internal

Deploy

Figure 6. Sales Aids -- customer-facing comparison generator.

MNST SALES ASSISTANT

Assistant
Customer Requirements
Discovery Questions
Sales Aids
Admin

Admin

Document management, index rebuilds, and claim policy editing.

Documents Approved Claims Restricted Claims Feedback Deals Customer Requirements

640 chunks indexed from 13 documents. Not rebuilt this session. Rebuild Index Now

Add a document

Upload a document to add it to the knowledge base.

Supported formats & extraction notes

Upload 200MB per file • CSV, DOCX, MARKDOWN, MD, PDF, PPTX, TXT, XLSX

Current documents

C1_Catalogue_FIXaHY-4220MA-RRNNV7II (PESO Approved Leak Detector Series).docx
205 KB • modified 2026-08-21 13:00
Product Catalogue Remove

Competitor Comparison_Hydrogen Sensor Performance.docx
38 KB • modified 2026-08-21 13:00
Competitive & Internal Strategy Remove

Stop Deploy

Figure 7. Admin -- Documents tab, knowledge base management.

How to Run the Project

Setup

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env
# then edit .env: at minimum set OPENAI_API_KEY and ADMIN_PASSWORD
```

Build the Index and Run

```
python -m app.rag_pipeline --reindex
streamlit run app/streamlit_app.py
```

Run Tests

```
pytest tests/
```

How to Modify the System

To change...	Go to...
Answer wording/tone/rules	app/prompts/system_prompt.md + answer_format.md
Chunking, product-name assignment	app/chunker.py
Retrieval scoping, top-k, thresholds	app/retriever.py (thresholds are env-overridable)
Deterministic catalog/decode routing	app/dispatcher.py
Restricted-claims policy (no code change)	Admin -> Restricted Claims tab
Approved Claims content (no code change)	Admin -> Approved Claims tab
Customer Requirements form fields	Admin -> Customer Requirements tab
UI look/theme	app/theme.py (CSS injection, applied unconditionally)
Which pages exist / nav order	app/streamlit_app.py, bottom of the file

Known Limitations and Edge Cases

- **No tie-breaking in intent classification** -- a question matching two categories silently resolves to whichever is listed first. Informational only, doesn't affect routing.
- **Claim-support checking is presence-based, not semantic** -- a source that mentions a certification term only to deny it could, in principle, misread as "supporting" evidence. Today's test cases pass, but the underlying mechanism isn't true entailment.
- **Two dead env vars** -- APP_MODE and CLAIM_GUARDRAIL_MODE are listed in .env.example but never read anywhere in app/. Setting them has no effect.
- **No access control on the four rep-facing pages** -- only Admin is password-gated; restricting the app's URL itself is a Streamlit Cloud sharing-settings concern.
- **Concurrent local Chroma access is unsafe** -- two Streamlit processes sharing one local index directory can corrupt it; recovery is delete-and-rebuild.
- **21 of the 36 evaluation-set questions have not been run and recorded** -- 15 have, with real results in reports/evaluation_report.xlsx (see Evaluation and Testing).
- **Warranty question gap, confirmed** -- asked directly, the app declines to state the FIXaHY series warranty duration instead of answering it, even though the evaluation set's own expected behavior assumes that's approved content. Reproduced across three separate runs.
- **Automated test coverage is narrow** -- the dispatcher, retrieval scoping, and the guardrail have no pytest coverage; verified manually through the real UI instead.

Troubleshooting

Symptom	Likely Cause	Fix
"Couldn't reach the database right now"	Neon's free-tier compute auto-suspended after inactivity and is waking up.	Wait ~30-60s and refresh. This is expected and handled gracefully, not a crash.
First question after a long idle period takes 1-2 minutes	Cold start -- pulling documents from Postgres, re-embedding, rebuilding the Product Registry.	Expected on the first visit after sleep. Warm the app up yourself before a demo.
"I do not have approved information for this" on a question that should be answerable	Retrieval similarity fell below the confidence threshold, or the document isn't indexed/approved yet.	Check Admin -> Documents that the source document is present and indexed; try Rebuild Index Now.
A just-deleted row reappears in a data table after saving	st.data_editor's stale row-diff, keyed by a fixed widget key.	Already fixed in code (widget key is bumped on every save) -- if seen again, confirm the deployed version is current.
An Admin confirm dialog doesn't close after clicking the action button	Known upstream Streamlit bug (#9405) -- a dialog won't close if real work runs before its own rerun.	Already worked around in code for all three dialogs; if reintroduced, keep dialog click handlers to "set a flag + rerun" only.
Customer-facing wording is blocked	The claim guardrail found a restricted-category term not backed by real source text -- working as intended.	Review the risk badge and source excerpts; escalate rather than override.
Two Streamlit processes running locally at once behave strangely	Concurrent access to the same local Chroma index directory.	Stop all but one process; if the index is corrupted, delete data/chroma_index/ and let it rebuild from Postgres or via --reindex.

Evaluation and Testing

Four pytest files, each a regression test written against a real, confirmed bug -- not a general coverage suite: chunking dilution in Approved Claims, table-header misdetection, XLSX title/table-region detection (including against the real production workbook), and the Postgres connection-retry path. Everything else -- the dispatcher, retrieval scoping, the claim guardrail, and every Streamlit page -- has been exercised by hand through the real UI rather than pytest; that manual testing happened extensively during development but wasn't logged as it went, so this document and the results below are the first written record of it.

15-question evaluation set

data/sample_questions.xlsx defines 36 questions across 10 categories (Product/Positioning, Application, Interfaces, Restricted Certification/Pricing/Delivery/Accuracy, Warranty, Unsupported/Out-of-scope, Customer-facing draft, Catalog/Deterministic, Comparison, a gibberish-input confidence check), each with a stated expected behavior. Summary of the 15:

Question	Result	Note
What products do you offer?	Pass	Correct deterministic catalog list, sources verified.
What is AURIGA?	Pass	Correct grounded description.
Is AURIGA a portable or fixed detector?	Pass	Correctly states portable.
Can the FIXaHY H2 LD integrate over RS485 or 4-20mA?	Pass	Confirms RS485; doesn't overclaim 4-20mA beyond what's documented.
Can AURIGA be deployed in a hazardous area?	Pass	Correctly refuses -- safe-area-only.
Does the FIXaHY H2 LD support wireless (BLE/LoRa)?	Review	Surfaces a customization-option caveat instead of a flat "not documented"; needs a manual check against the source text.
Is the FIXaHY-4220MA PESO approved for hazardous areas?	Pass	Correct, model-specific (Zone 1 IIC), not over-generalized to other SKUs.
Is AURIGA ATEX certified?	Pass	Correctly declines -- not documented.
What is the price of AURIGA?	Pass	Correctly declines -- no price data in approved docs.
What is the warranty on the FIXaHY series?	Fail	GAP -- declines instead of stating a duration, though the eval set assumes this is approved content. Reproduced 3x.
Do you offer a device for detecting carbon monoxide?	Pass	Correctly declines -- H2-only.
Draft a customer-facing reply re: battery-room H2 detection.	Pass	Grounded draft generated end-to-end via the customer-wording button.
List all products you offer.	Pass	Same correct list as Q1, phrasing-independent.
Compare the FIXaHY H2 LD and PORTaHY H2 LD.	Pass	Balanced table comparison, not dominated by either product.

(gibberish input)	Pass	Declines gracefully -- no error, no hallucination.
-------------------	------	--

The Admin Feedback tab computes a "Feedback Accuracy" percentage from real rep Correct/Wrong/Unsafe clicks -- explicitly caveated in the UI itself as reflecting only rated answers, not a random sample.

Final Handover Checklist

- ☐ Can create a .env from .env.example and get the app running locally end-to-end.
- ☐ Has access to the Neon Postgres project and understands DATABASE_URL must be the pooled connection string.
- ☐ Has access to the Streamlit Community Cloud deployment and its Secrets configuration.
- ☐ Understands the dual-write pattern (Postgres vs. local-file fallback) in every *_store.py module.
- ☐ Has read the st.dialog close-bug workaround before touching any Admin confirmation dialog.
- ☐ Understands product_registry.json is fully regenerated on every reindex -- never hand-edit it.
- ☐ Has run pytest tests/ successfully and understands what it does and doesn't cover.
- ☐ Understands "Remove" in Admin moves a document to data/removed_docs/ -- it does not delete permanently.
- ☐ Has reviewed the GitHub Actions workflows that are required for the deployed application and confirmed any required repository secrets are available to the company.
- ☐ Has personally asked real questions through the live Assistant UI and confirmed the full pipeline works.